

REPETIÇÕES E ESTRUTURAS DE DADOS

REPETINDO TAREFAS SEM PRECISAR REPETIR CÓDIGO



Caroline Perez Gonçalves
2026

REPETIÇÕES E ESTRUTURAS DE DADOS

Caroline Perez Gonçalves

SUMÁRIO

APRESENTAÇÃO.....	1
INTRODUÇÃO	2
LAÇO WHILE.....	3
LAÇO FOR.....	6
FUNÇÃO RANGE ().....	10
LISTAS.....	14
TUPLAS	18
DICIONÁRIOS	20
CONJUNTOS (SETS)	24
PERCORRENDO COLEÇÕES	27
PROJETO PRÁTICO.....	31
AGRADECIMENTO.....	35
CONTATO.....	36

APRESENTAÇÃO

Aos que não me conhecem, meu nome é Caroline. Sou formada em Análise e Desenvolvimento de Sistemas, pós-graduada em Inteligência Artificial e Machine Learning e possuo MBA em Gerenciamento de Projetos.

Além da minha experiência profissional com tecnologia, projetos e equipes multidisciplinares, também atuo como instrutora. Sempre acreditei que o conhecimento deve ser compartilhado de forma simples, leve e acessível, e foi com esse propósito que esta apostila foi desenvolvida.

O intuito deste material é mostrar que o Python está presente em muito mais situações do que imaginamos e que entender essa tecnologia não precisa ser algo complicado. Muito mais do que aprender ferramentas, o mais importante é compreender os conceitos, entender como a linguagem funciona, onde ela pode ser aplicada e como ela pode transformar as coisas.

Ao final desta apostila, espero que você tenha construído uma base sólida sobre os fundamentos dessa linguagem e se sinta mais confiante para acompanhar as próximas trilhas de estudo e explorar esse universo com mais segurança.

Vamos aos estudos! Mas, antes, sente-se, relaxe e sinta-se em casa. Quero que este material seja como uma boa conversa entre amigos, tornando o aprendizado mais leve, agradável e, principalmente, significativo.

Boa sorte e muito sucesso!

INTRODUÇÃO

Afinal, por que aprender Python?

A resposta para essa e outras perguntas tem um segredo. Muita gente acredita que programação é um assunto complicado, cheio de códigos difíceis ou exclusivo para quem trabalha com tecnologia. Mas a verdade é justamente o contrário.

Como assim, Carol?

A programação está presente em praticamente tudo o que fazemos. Ela está por trás dos aplicativos que usamos diariamente, dos sites que acessamos, dos sistemas bancários, das compras online, dos aplicativos de transporte, das redes sociais e até de muitos equipamentos que fazem parte da nossa rotina.

Quando você entende isso, percebe que aprender Python não significa apenas estudar uma linguagem de programação. Significa desenvolver uma nova forma de pensar, aprender a resolver problemas de maneira lógica e criar soluções para situações do dia a dia.

E tem mais: aprender Python não é decorar comandos. Os comandos podem até mudar de uma linguagem para outra, mas o que realmente faz diferença é compreender os conceitos, desenvolver o raciocínio lógico e entender como construir programas capazes de automatizar tarefas e facilitar o trabalho das pessoas.

Hoje o Python é utilizado em diversas áreas, como desenvolvimento de sistemas, automação, análise de dados, Inteligência Artificial, desenvolvimento Web, computação em nuvem, segurança da informação e muitas outras. Isso faz dele uma das linguagens mais versáteis e procuradas do mercado.

Claro, existem assuntos mais avançados que exigem bastante estudo e prática. Mas, como em qualquer aprendizado, tudo começa pelos fundamentos. E é exatamente isso que vamos construir ao longo desta apostila.

Para que essa introdução não fique muito extensa, vamos explorar cada tema no momento certo. E, como todo bom aprendizado, vamos começar pelo começo.

Vamos lá.

LAÇO WHILE

Imagine que o sistema que você desenvolveu para cadastro de clientes está funcionando. Depois de cadastrar o primeiro cliente, o programa é encerrado.

Se chegar um segundo cliente, será necessário executar o programa novamente.

Depois um terceiro.

Depois um quarto.

Não é muito prático, né?

Agora imagine uma loja com centenas de clientes sendo cadastrados todos os dias.

Executar o programa manualmente a cada novo cadastro seria um enorme desperdício de tempo. É justamente para resolver esse tipo de situação que existem os **laços de repetição**.

Eles permitem que um mesmo bloco de código seja executado várias vezes de forma automática.

Nesta primeira parte vamos falar sobre o **while**.

O que significa while?

A palavra **while**, em inglês, significa "**enquanto**".

A ideia é muito simples.

Enquanto uma determinada condição for verdadeira, o programa continuará executando aquele bloco de código. Quando essa condição deixar de ser verdadeira, a repetição será encerrada. Veja um exemplo.

```
contador = 1
```

```
while contador <= 5:
```

```
    print(contador)
```

```
    contador += 1
```

Resultado:

1

2

3

4

5

Observe o que aconteceu. O programa começou com a variável **contador** valendo 1. Enquanto o contador fosse menor ou igual a 5, o Python executava o bloco de código. Ao final de cada repetição, somamos 1 ao contador.

Quando ele chegou ao valor 6, a condição deixou de ser verdadeira e o laço foi encerrado.

Como o while funciona?

Sempre que o Python encontra um **while**, ele faz uma pergunta: “ a condição ainda é verdadeira?”

Se a resposta for **sim**, ele executa o bloco de código. Quando termina, faz a mesma pergunta novamente.

Se a condição continuar verdadeira, executa tudo outra vez. Esse processo continua até que a resposta seja **não**. É exatamente por isso que o laço recebe o nome **while**.

Enquanto a condição for verdadeira, ele continuará repetindo. Uma, duas, três, quatro vezes ou ENQUANTO a condição for verdadeira.

Um exemplo do dia a dia

Imagine uma máquina de senha em um consultório médico. Aquelas que fica chamando senha e nomes no painel.

Enquanto ainda existirem pacientes aguardando atendimento, a recepcionista continuará chamando novas senhas. Quando não houver mais ninguém esperando, o atendimento é encerrado.

Esse funcionamento é muito parecido com um laço **while**.

Enquanto existir algo para fazer, o programa continua trabalhando.

A importância do contador

Observe novamente este trecho.

```
contador += 1
```

Essa linha é extremamente importante. Ela faz com que o valor do contador aumente a cada repetição. Sem ela, a condição nunca mudaria. Veja este exemplo.

```
contador = 1
```

```
while contador <= 5:
```

```
    print(contador)
```

O que acontecerá?

O contador continuará valendo 1 para sempre. Como a condição nunca deixará de ser verdadeira, o programa ficará preso em uma repetição infinita.

Esse tipo de problema recebe o nome de **loop infinito**. E certamente vai acontecer em algum momento no seu código.

O que é um loop infinito?

Um loop infinito acontece quando a condição do **while** nunca deixa de ser verdadeira.

Veja novamente o exemplo anterior.

```
contador = 1
```

```
while contador <= 5:
```

```
    print(contador)
```

Como o contador nunca é alterado, o programa continuará exibindo:

1

1

1

1

1

1

...

Sem parar. Para todo sempre. E em programas reais isso pode causar diversos problemas. Por isso, sempre que utilizar um **while**, pergunte a si mesmo:

"Em que momento essa condição deixará de ser verdadeira?"

Se você não conseguir responder essa pergunta, provavelmente existe um erro no seu código. Se você rodar, vai travar, você vai achar que foi o fim do código, mas é só um loop eterno.

Utilizando while com o usuário

O laço **while** também pode ser utilizado para repetir perguntas.

Imagine um sistema de login. Enquanto a senha estiver incorreta, o programa continuará solicitando uma nova tentativa. Veja um exemplo.


```
senha = ""

while senha != "python123":

    senha = input("Digite a senha: ")

print("Acesso liberado.")
```

Perceba como o programa continua perguntando a senha até que o usuário informe o valor correto. Enquanto a senha digitada for diferente de python123, quero exibir na tela “Digite a senha: “.

Esse tipo de comportamento aparece em diversos sistemas que utilizamos diariamente.

Outro exemplo

Imagine um caixa eletrônico. Enquanto o cliente desejar realizar novas operações, o sistema continuará funcionando.

Somente quando o usuário escolher a opção de sair o programa será encerrado. Esse é outro exemplo clássico de utilização do **while**.

Quando utilizar o while?

O laço **while** é indicado quando **não sabemos exatamente quantas vezes uma repetição acontecerá**. Alguns exemplos.

- Solicitar uma senha até que ela esteja correta;
- Validar um CPF informado pelo usuário;
- Exibir um menu até que o usuário escolha sair;
- Receber vários cadastros;
- Continuar um atendimento enquanto houver clientes.

Sempre que a repetição depender de uma condição, o **while** costuma ser uma excelente escolha.

LAÇO FOR

Ele é uma excelente opção quando a repetição depende de uma condição. Mas imagine outra situação.

Você precisa exibir na tela os números de 1 até 10 ou mostrar todos os produtos cadastrados ou, ainda, percorrer uma lista de clientes. Será que faz sentido criar um contador manualmente toda vez?

Nem sempre.

Para situações como essas, o Python oferece uma estrutura muito prática chamada **for**.

O laço **for** é utilizado quando queremos percorrer uma sequência de informações ou repetir uma tarefa uma quantidade conhecida de vezes.

Na prática, ele acaba sendo um dos comandos mais utilizados da linguagem.

O que significa for?

A palavra **for**, em inglês, significa algo como "**para cada**".

Imagine uma caixa contendo dez documentos e você precisa ler todos eles. Em vez de pegar cada documento manualmente, você pode pensar da seguinte forma.

Para cada documento da caixa, faça a leitura.

É exatamente essa ideia que o **for** utiliza. Ele percorre uma sequência, executando o mesmo bloco de código para cada elemento encontrado.

Primeiro exemplo

Vamos utilizar o **for** para exibir os números de 1 até 5.

```
for numero in range(1, 6):  
    print(numero)
```

Resultado:

1
2
3
4
5

Observe que não precisamos criar um contador. O próprio **for** controla a repetição automaticamente.

Isso deixa o código muito mais simples e organizado.

Entendendo o exemplo

Observe novamente o exemplo:

for numero in range(1, 6):

Vamos dividir essa linha em partes.

for = indica que iniciaremos um laço de repetição.

numero = é uma variável criada pelo próprio laço.

A cada repetição ela receberá um novo valor.

In = pode ser entendido como "**em**".

Ou seja:

Para cada número **na sequência**...

range(1, 6)

Cria uma sequência iniciando em 1 e terminando antes do 6.

Por isso o último número exibido será 5.

Outro exemplo

Imagine que você queira imprimir uma mensagem cinco vezes.

for numero in range(5):

```
    print("Bem-vindo ao sistema!")
```

Resultado:

Bem-vindo ao sistema!

Bem-vindo ao sistema!

Bem-vindo ao sistema!

Bem-vindo ao sistema!

Bem-vindo ao sistema!

Perceba que nem utilizamos a variável **numero**. Ela existe, mas neste exemplo não é necessária. O que importa é que exibimos a mensagem 5 vezes.

For ou While?

Essa é uma dúvida muito comum entre quem está começando. Quando utilizar cada um deles?

De forma geral, podemos pensar assim:

Utilize **while** quando a repetição depender de uma condição. Por exemplo.

- Enquanto a senha estiver incorreta;
- Enquanto houver clientes aguardando atendimento;
- Enquanto o usuário desejar continuar.

Utilize **for** quando souber aproximadamente quantas vezes a repetição acontecerá. Por exemplo.

- Exibir os números de 1 até 100;
- Percorrer uma lista de produtos;
- Mostrar todos os alunos cadastrados.

Não existe uma regra absoluta, mas essa diferença ajuda bastante no início.

O for também percorre textos

Uma característica interessante do **for** é que ele pode percorrer letras de uma palavra. Veja um exemplo.

```
nome = "Python"
```

```
for letra in nome:
```

```
    print(letra)
```

Resultado:

P

y

t

h

o

n

Perceba que o laço percorreu cada caractere da palavra.

Observação: mais adiante veremos que ele também pode percorrer listas, tuplas, conjuntos e dicionários.

Outro exemplo do dia a dia

Imagine um sistema que envia e-mails para todos os clientes cadastrados. O programa

precisa fazer exatamente a mesma tarefa para cada cliente, que é enviar a mensagem.

Não importa se existem dez ou dez mil clientes.

O comportamento será o mesmo e é exatamente esse tipo de situação que torna o **for** tão útil.

Vantagens do for

- O código fica mais organizado;
- Não precisamos controlar um contador manualmente;
- A leitura se torna mais simples;
- Reduzimos a chance de criar loops infinitos;
- Facilita o trabalho com grandes quantidades de informações.

FUNÇÃO RANGE ()

Nós acabamos de utilizar laço **for** para repetir uma tarefa várias vezes e eu não sei se você percebeu, mas temos um elemento diferente nesse código. Vou repetir uma linha do código só para facilitar o entendimento.

for numero in range(1, 6):

A função **range()** está ali. Mas afinal...

O que ela faz?

A função **range()** é responsável por criar uma sequência de números.

Essa sequência pode ser utilizada em diversas situações, mas seu uso mais comum é controlar quantas vezes um laço **for** será executado.

Em outras palavras, o **for** realiza a repetição e o **range()** informa quais valores serão percorridos durante essa repetição.

Embora eles sejam frequentemente utilizados juntos, cada um possui uma função diferente.

Primeiro exemplo

Veja novamente o código.

for numero in range(1, 6):

```
print(numero)
```

Resultado:

```
1
2
3
4
5
```

Observe um detalhe importante. Embora o número **6** tenha sido informado na função `range()`, ele não foi exibido. Isso acontece porque o último valor informado representa o limite da sequência.

Ou seja, o `range()` sempre para **antes** do último número.

Esse comportamento costuma gerar dúvidas no início. Então, não se incomode com essa sensação.

Utilizando apenas um parâmetro

A forma mais simples de utilizar o `range()` é informando apenas um número. Veja um exemplo.

```
for numero in range(5):
```

```
    print(numero)
```

Resultado:

```
0
1
2
3
4
```

Perceba que a sequência começou em **0**.

Sempre que utilizamos apenas um parâmetro, o `range()` inicia automaticamente no zero. Esse comportamento é muito comum em programação. Então, quando trabalharmos listas, arrays ou outros tipos de coleções, o primeiro elemento possui o índice 0,, o segundo possui índice 1, e assim por diante.

Definindo início e fim

Também podemos informar onde a sequência deve começar. Veja um exemplo.

```
for numero in range(3, 8):  
    print(numero)
```

Resultado:

3
4
5
6
7

Mais uma vez, o número final não foi exibido. O valor **8** apenas indica onde a sequência deve terminar.

Definindo o intervalo

Além do início e do fim, podemos informar de quanto em quanto a sequência deverá avançar. Veja um exemplo.

```
for numero in range(2, 11, 2):  
    print(numero)
```

Resultado:

2
4
6
8
10

Observe que agora os números aumentam de dois em dois.

Outro exemplo.

```
for numero in range(5, 26, 5):  
    print(numero)
```

Resultado:

5
10
15
20
25

Esse recurso é muito útil quando precisamos trabalhar com intervalos específicos.

Contagem regressiva

O `range()` também permite criar sequências decrescentes.

Para isso utilizamos um incremento negativo. Veja um exemplo.

```
for numero in range(10, 0, -1):  
    print(numero)
```

Resultado:

10
9
8
7
6
5
4
3
2
1

Perceba que agora a sequência acontece de trás para frente. Esse tipo de repetição aparece em diversas situações.

- Temporizadores;
- Jogos;
- Sistemas de monitoramento.

Um exemplo do dia a dia

Imagine um sistema que envia mensagens para os cinco primeiros clientes cadastrados. Ou um programa que imprime etiquetas numeradas de 1 até 100.

Em ambos os casos precisamos gerar uma sequência de números e é exatamente esse tipo de tarefa que a função `range()` realiza.

- Ela cria a sequência.
- O laço `for` percorre essa sequência.

Quando utilizar o `range()`?

Sempre que precisarmos trabalhar com uma sequência numérica. Alguns exemplos.

- Contar de 1 até 100;
- Exibir uma contagem regressiva;
- Repetir uma tarefa determinada quantidade de vezes;
- Percorrer posições de uma coleção;
- Criar intervalos numéricos.

Na prática, você vai perceber que o `range()` aparece frequentemente ao lado do `for`.

LISTAS

Até agora, sempre que precisávamos armazenar uma informação, criávamos uma variável. Vamos lembrar:

```
cliente1 = "Ana"
```

```
cliente2 = "Carlos"
```

```
cliente3 = "Fernanda"
```

```
cliente4 = "Marcos"
```

Esse código funciona. Mas imagine uma empresa que possui cinco mil clientes cadastrados. Será que criar cinco mil variáveis seria uma boa ideia? Eu sinto que não (claro que não!).

Além de trabalhoso, o programa ficaria extremamente difícil de manter. É justamente para resolver esse problema que existem as **listas**.

Uma lista permite armazenar vários valores em uma única variável.

Em vez de criar dezenas ou centenas de variáveis diferentes, organizamos todas

essas informações dentro de uma única estrutura.

Pense em uma lista de compras. Você não cria uma folha diferente para cada produto. Todos eles ficam organizados na mesma lista, uma embaixo da outra (ou riscada do lado caso você esqueça de algo rs).

Em Python acontece exatamente a mesma coisa.

Criando uma lista

Para criar uma lista utilizamos colchetes []. Veja um exemplo.

```
clientes = ["Ana", "Carlos", "Fernanda", "Marcos"]
```

Observe que agora todos os nomes estão armazenados em uma única variável chamada **clientes**. Muito mais organizado.

Uma lista pode armazenar praticamente qualquer tipo de dado. Por exemplo.:

```
idades = [18, 25, 32, 41]
```

```
precos = [29.90, 159.50, 980.00]
```

Também podemos misturar diferentes tipos de dados.

```
dados = ["Caroline", 32, True, 1899.90]
```

Embora isso seja permitido, na maioria das situações é recomendado que a lista contenha informações relacionadas entre si.

Isso torna o código mais organizado e mais fácil de entender.

Acessando um elemento da lista

Imagine uma estante com vários livros. Cada livro ocupa uma posição.

Em uma lista acontece algo parecido.

Cada elemento possui uma posição chamada **índice**. Veja o exemplo.

```
clientes = ["Ana", "Carlos", "Fernanda", "Marcos"]
```

As posições ficam assim.

Índice:	0	1	2	3
---------	---	---	---	---

Valor:	Ana	Carlos	Fernanda	Marcos
--------	-----	--------	----------	--------

Em Python, a contagem começa no **zero**.

Esse é um comportamento muito comum em linguagens de programação.

Para acessar um elemento específico utilizamos seu índice.

```
print(clientes[0])
```

Resultado: Ana

Outro exemplo.

```
print(clientes[2])
```

Resultado: Fernanda

Alterando um elemento

Também podemos modificar informações já existentes. Veja um exemplo:

```
clientes = ["Ana", "Carlos", "Fernanda"]
```

Agora vamos substituir o primeiro nome:

```
clientes[0] = "Juliana"
```

A lista ficará assim:

```
["Juliana", "Carlos", "Fernanda"]
```

Esse recurso é bastante utilizado quando precisamos atualizar informações.

Adicionando novos elementos

Uma lista também pode crescer conforme o programa é executado.

Para adicionar um novo elemento utilizamos o método **append()**.

Veja um exemplo.

```
clientes = ["Ana", "Carlos"]
```

```
clientes.append("Fernanda")
```

Resultado: ["Ana", "Carlos", "Fernanda"]

Sempre que utilizamos **append()**, o novo elemento é inserido no final da lista.

Removendo elementos

Também podemos remover informações. Veja um exemplo.

```
clientes = ["Ana", "Carlos", "Fernanda"]
```

```
clientes.remove("Carlos")
```

Resultado: ["Ana", "Fernanda"]

Esse recurso é muito utilizado quando um cadastro é excluído ou um produto deixa de fazer parte de uma coleção.

Descobrimos quantos elementos existem

Imagine um sistema que precisa informar quantos clientes estão cadastrados. Para isso utilizamos a função **len()**. Veja um exemplo.

```
clientes = ["Ana", "Carlos", "Fernanda", "Marcos"]
```

```
print(len(clientes))
```

Resultado: 4

A função **len()** retorna a quantidade de elementos presentes na lista.

Um exemplo do dia a dia

Imagine um supermercado. Todos os produtos vendidos precisam ser armazenados pelo sistema. Seria inviável criar uma variável para cada produto. Em vez disso, utilizamos listas.

O mesmo acontece com:

- Clientes;
- Funcionários;
- Pedidos;
- Produtos;
- Notas fiscais;
- Veículos;

Sempre que precisamos trabalhar com várias informações semelhantes, uma lista costuma ser uma excelente escolha.

Quando utilizar listas?

As listas são indicadas sempre que precisarmos armazenar uma coleção de informações. Alguns exemplos.

- Lista de alunos;
- Lista de produtos;
- Lista de clientes;
- Lista de cidades;
- Lista de preços;

Em praticamente qualquer sistema existe alguma situação em que uma lista será utilizada.

TUPLAS

Vimos sobre listas, que elas permitem armazenar vários elementos em uma única variável e podem ser modificadas sempre que necessário. Podemos adicionar novos itens, remover elementos e alterar informações.

Mas imagine se estamos trabalhando com um sistema que controla os meses do ano. Será que faz sentido permitir que alguém remova o mês de fevereiro? Ou adicione um décimo terceiro mês?

Eu acredito que não.

Essas informações nunca mudam. É justamente para esse tipo de situação que existem as **tuplas**.

Uma tupla é uma estrutura muito parecida com uma lista. A principal diferença é que, depois de criada, **ela não pode ser alterada**.

Essa característica torna as tuplas mais seguras para armazenar informações que devem permanecer sempre iguais durante a execução do programa.

Criando uma tupla

Enquanto as listas utilizam colchetes [], as tuplas utilizam parênteses (). Veja um exemplo.

```
meses = (
```

```
    "Janeiro",
```

```
    "Fevereiro",
```

```
    "Março",
```

```
    "Abril"
```

```
)
```

Observe que a estrutura é bastante parecida com uma lista. A diferença está apenas nos parênteses.

Acessando elementos

Assim como acontece nas listas, cada elemento possui uma posição.

Veja o exemplo.

```
meses = (
```

```
    "Janeiro",
```

```
    "Fevereiro",
```

```
    "Março",
```

```
    "Abril"
```

```
)
```

```
print(meses[0])
```

Resultado: Janeiro

Outro exemplo.

```
print(meses[2])
```

Resultado: Março

Assim como nas listas, a contagem começa no índice **0**.

O que não podemos fazer?

Agora veja o exemplo abaixo.

```
meses[0] = "Junho"
```

O Python apresentará um erro. Isso acontece porque uma tupla é **imutável**. Precisei seguir esse padrão, porque é exatamente assim que você vai encontrar em outras explicações. Imutável significa que, depois de criada, seus elementos não podem ser alterados.

Também não é possível adicionar ou remover elementos.

Esse comportamento é proposital. Ele evita alterações acidentais em informações que deveriam permanecer constantes.

Quando utilizar uma tupla?

Sempre que os dados não precisarem ser modificados. Alguns exemplos:

- Meses do ano;
- Dias da semana;
- Estados brasileiros;
- Categorias de produtos;
- Configurações que não devem mudar.

Imagine um sistema que armazena os dias da semana.

```
dias_semana = (  
    "Segunda-feira",  
    "Terça-feira",  
    "Quarta-feira",  
    "Quinta-feira",  
    "Sexta-feira",  
    "Sábado",  
    "Domingo"  
)
```

Faz sentido alguém adicionar uma segunda "sexta-feira" ou remover o domingo?

Lista ou tupla?

Essa é uma dúvida muito comum e a resposta depende da necessidade do programa.

Utilize uma **lista** quando os dados puderem mudar. Já a **tupla** deve ser utilizada quando os dados permanecerão fixos.

Em outras palavras, se existe a possibilidade de alteração, normalmente utilizamos uma lista. Se os dados permanecerão constantes, uma tupla costuma ser a melhor opção.

DICIONÁRIOS

Até agora conhecemos duas importantes estruturas de dados: listas e tuplas. Agora, os dicionários armazenam pares chave-valor. Conseguiu entender?

Quando eu li pela primeira vez, eu também não entendi. Então, vamos para a explicação mais fácil. Vamos lá imaginar...

Você está desenvolvendo um sistema de cadastro de clientes e cada cliente possui diversas informações:

- Nome;
- Idade;
- Cidade;
- Telefone;
- E-mail.

Poderíamos armazenar tudo isso em uma lista? Poderíamos.

Mas imagine precisar descobrir qual posição representa o telefone. Ou em qual índice está armazenada a cidade. O código rapidamente ficaria difícil de entender.

É justamente para resolver esse problema que existem os **dicionários**.

Diferentemente das listas e das tuplas, os dicionários organizam as informações utilizando uma relação entre **chave** e **valor**.

Pense em um dicionário da língua portuguesa. Quando você procura a palavra **computador**, encontra sua definição.

Nesse caso: a palavra é a chave e a definição é o valor.

Em Python acontece algo parecido. Cada informação recebe uma identificação que facilita sua localização.

Criando um dicionário

Os dicionários utilizam chaves {}.

Veja um exemplo.

```
cliente = {  
    "nome": "Caroline",  
    "idade": 32,  
    "cidade": "São Paulo",  
    "telefone": "(11) 99999-9999"  
}
```

Observe que cada informação possui uma chave.

- "nome" → "Caroline";
- "idade" → 32;
- "cidade" → "São Paulo";
- "telefone" → "(11) 99999-9999".

Isso torna o código muito mais fácil de compreender.

Acessando informações

Para acessar um valor, basta informar sua chave. Veja um exemplo:

```
print(cliente["nome"])
```

Resultado: Caroline

Outro exemplo:

```
print(cliente["cidade"])
```

Resultado: São Paulo

Perceba que não precisamos lembrar posições como acontece nas listas. Basta informar o nome da informação que desejamos acessar e ela vem.

Alterando um valor

Também podemos modificar informações já existentes. Veja um exemplo:

```
cliente["cidade"] = "Curitiba"
```

Agora o dicionário ficará assim.

```
{  
    "nome": "Caroline",  
    "idade": 32,  
    "cidade": "Curitiba",  
    "telefone": "(11) 99999-9999"  
}
```

Adicionando uma nova informação

Os dicionários também permitem adicionar novos dados. Veja um exemplo:

```
cliente["email"] = "caroline@email.com"
```

Resultado.

```
{  
  "nome": "Caroline",  
  "idade": 32,  
  "cidade": "Curitiba",  
  "telefone": "(11) 99999-9999",  
  "email": "caroline@email.com"  
}
```

Observe como o novo campo foi incluído de forma bastante simples.

Removendo informações

Também é possível remover um item do dicionário. Veja um exemplo.

```
del cliente["telefone"]
```

Agora o telefone deixará de fazer parte do cadastro.

Um exemplo do dia a dia

Imagine um sistema hospitalar, onde cada paciente possui diversas informações.

- Nome;
- CPF;
- Data de nascimento;
- Convênio;
- Tipo sanguíneo;
- Telefone.

Seria muito mais fácil acessar **paciente["cpf"]** do que tentar lembrar que o CPF está na posição 4 de uma lista.

É justamente por isso que os dicionários são tão utilizados.

Quando utilizar um dicionário?

Os dicionários são indicados sempre que uma informação possuir características bem definidas. Alguns exemplos são os que já vimos acima: cadastro de clientes, produtos, funcionários, veículos, pedidos, usuários, etc.

Sempre que precisarmos associar um nome a um valor, um dicionário costuma ser uma excelente escolha.

Lista ou dicionário?

Essa é outra dúvida bastante comum e nós podemos pensar da seguinte forma.

Utilize uma **lista** quando o mais importante for a sequência de elementos. Por exemplo, lista de cidades, lista de produtos ou lista de alunos.

Agora imagine um único aluno. Esse aluno possui várias características, por exemplo: nome, idade, matrícula e curso.

Nesse caso, um **dicionário** faz muito mais sentido.

No resumo: uma lista costuma armazenar **vários elementos semelhantes**. Já um dicionário normalmente representa **um único elemento com diversas características**.

CONJUNTOS (SETS)

As três importantes estruturas que conhecemos até agora foram:

As listas armazenam vários elementos e permitem alterações.

As tuplas armazenam vários elementos, mas não podem ser modificadas.

Os dicionários organizam informações utilizando chaves e valores.

Agora imagine outra situação (tá, confesso que eu também já estou ficando sem imaginação). Mas, você está desenvolvendo um sistema para um evento.

Durante o cadastro, algumas pessoas acabam sendo registradas mais de uma vez. Veja um exemplo.

- Ana;
- Carlos;
- Fernanda;
- Ana;

- Marcos;
- Carlos.

Percebe o problema? Existem nomes repetidos.

Se o objetivo for apenas descobrir quem está inscrito no evento, essas duplicidades não fazem sentido.

É justamente para resolver situações como essa que existem os **conjuntos**, conhecidos em Python como **sets**.

Um conjunto armazena apenas **elementos únicos**.

Se um mesmo valor for adicionado mais de uma vez, ele será armazenado apenas uma única vez.

Criando um conjunto

Os conjuntos utilizam chaves {}, assim como os dicionários.

A diferença é que, nos conjuntos, armazenamos apenas os valores. Veja um exemplo:

```
clientes = {"Ana", "Carlos", "Fernanda", "Marcos"}
```

Observe que não existem chaves e valores. Aqui temos apenas os elementos do conjunto.

Valores repetidos

Agora veja o exemplo abaixo.

```
clientes = {"Ana", "Carlos", "Ana", "Fernanda", "Carlos"}
```

Resultado:

```
{"Ana", "Carlos", "Fernanda"}
```

Mesmo informando "Ana" e "Carlos" duas vezes, o conjunto armazenou apenas um exemplar de cada elemento.

Essa é a principal característica.

Adicionando novos elementos

Também podemos incluir novos valores. Para isso utilizamos o método **add()**. Veja um exemplo:

```
clientes = {"Ana", "Carlos"}
```

```
clientes.add("Fernanda")
```

Resultado: {"Ana", "Carlos", "Fernanda"}

Agora imagine adicionar novamente o mesmo nome.

```
clientes.add("Carlos")
```

Nada vai acontecer. O conjunto continuará armazenando apenas um único "Carlos".

Removendo elementos

Também podemos remover informações. Veja um exemplo:

```
clientes.remove("Ana")
```

Resultado: {"Carlos", "Fernanda"}

Uma característica super importante

Ao contrário das listas, os conjuntos **não possuem uma ordem definida**.

Isso significa que os elementos podem ser exibidos em uma ordem diferente daquela utilizada na criação do conjunto. Por exemplo:

```
clientes = {"Ana", "Carlos", "Fernanda"}
```

Ao exibir esse conjunto, o resultado poderá ser:

```
{"Carlos", "Ana", "Fernanda"}
```

Ou outra ordem qualquer.

Isso acontece porque o objetivo do conjunto não é organizar elementos por posição.

Seu principal objetivo é garantir que não existam valores duplicados.

Por esse motivo, **não** podemos acessar elementos utilizando índices, como fazemos nas listas.

Um exemplo do dia a dia

Imagine um sistema de uma empresa.

Durante o dia, centenas de acessos são registrados. O objetivo é descobrir apenas quais funcionários acessaram o prédio.

Não importa quantas vezes cada um entrou. O que importa é saber quem esteve presente.

Nesse caso, um conjunto é uma excelente escolha.

Agora, perguntinha marota.

Lista ou conjunto?

Essa é outra dúvida bastante comum.

Utilize **lista** quando a ordem dos elementos for importante, valores repetidos forem permitidos ou que o acesso dos elementos seja pelo índice.

Utilize um **conjunto** quando não desejarmos elementos repetidos, a ordem dos elementos não for importante ou quando precisarmos garantir que cada informação apareça apenas uma vez.

Cada estrutura foi criada para resolver um tipo diferente de problema.

PERCORRENDO COLEÇÕES

Como acessar todos esses elementos sem escrever uma linha de código para cada um deles?

Vai ser a última vez que vou pedir que você imagine algo (não prometo). Mas, pense em uma loja com mil produtos cadastrados.

Ou uma escola com centenas de alunos.

Ou uma empresa com milhares de clientes.

Será que seria necessário escrever um `print()` para cada informação?

A resposta é não.

É justamente para isso que utilizamos os laços de repetição. Eles permitem percorrer todos os elementos de uma coleção automaticamente.

Na maioria das situações utilizaremos o laço **for**.

Percorrendo uma lista

Vamos começar com o exemplo mais comum.

Imagine uma lista contendo alguns clientes.

```
clientes = ["Ana", "Carlos", "Fernanda", "Marcos"]
```

Para percorrer todos os elementos utilizamos o `for`.

```
clientes = ["Ana", "Carlos", "Fernanda", "Marcos"]
```

```
for cliente in clientes:
```

```
    print(cliente)
```

Resultado:

Ana

Carlos

Fernanda

Marcos

Observe como o código ficou simples.

A cada repetição, a variável **cliente** recebe um novo elemento da lista. Quando todos os elementos forem percorridos, o laço será encerrado automaticamente.

Percorrendo uma tupla

O processo é praticamente o mesmo. Veja um exemplo:

```
dias_semana = (
```

```
    "Segunda",
```

```
    "Terça",
```

```
    "Quarta",
```

```
    "Quinta",
```

```
    "Sexta"
```

```
)
```

```
for dia in dias_semana:
```

```
    print(dia)
```

Resultado:

Segunda

Terça

Quarta

Quinta

Sexta

Perceba que pouco mudou. O for consegue percorrer diferentes tipos de coleções da mesma forma.

Percorrendo um conjunto

Os conjuntos também podem ser percorridos. Veja um exemplo:

```
categorias = {  
    "Eletrônicos",  
    "Livros",  
    "Roupas",  
    "Brinquedos"  
}
```

```
for categoria in categorias:
```

```
    print(categoria)
```

O resultado exibirá todas as categorias.

Mas existe um detalhe importante!

Como vimos anteriormente, os conjuntos não possuem uma ordem definida.

Por isso, os elementos poderão aparecer em uma sequência diferente daquela utilizada na criação do conjunto e isso é completamente normal.

Percorrendo um dicionário

Os dicionários funcionam de maneira um pouco diferente.

Quando utilizamos um for diretamente sobre um dicionário, percorremos suas chaves. Veja um exemplo:

```
cliente = {  
    "nome": "Caroline",
```



```
"idade": 32,  
"cidade": "São Paulo"  
}
```

for dado in cliente:

```
    print(dado)
```

Resultado:

nome

idade

cidade

Observe que foram exibidas apenas as chaves. Mas e se quisermos visualizar também os valores?

Para isso utilizamos o método **items()**.

```
cliente = {  
    "nome": "Caroline",  
    "idade": 32,  
    "cidade": "São Paulo"  
}
```

for chave, valor in cliente.items():

```
    print(chave, ":", valor)
```

Resultado:

nome : Caroline

idade : 32

cidade : São Paulo

Esse recurso é bastante utilizado em sistemas que trabalham com cadastros e configurações.

PROJETO PRÁTICO

Sistema de Cadastro de Alunos

Objetivo: desenvolver um programa capaz de cadastrar vários alunos e exibir um relatório ao final da execução.

Eu menti quando disse que não ia pedir mais isso... Mas, imagine que uma escola precisa cadastrar os alunos matriculados em um curso. Cada aluno deverá possuir:

- Nome;
- Idade;
- Cidade.

O programa deverá permitir o cadastro de vários alunos.

Quando o usuário decidir encerrar os cadastros, será exibido um relatório contendo todas as informações registradas.

Desafio

Desenvolva um programa que:

- 1 Solicite o nome do aluno;
- 2 Solicite a idade;
- 3 Solicite a cidade;
- 4 Armazene cada aluno em um dicionário;
- 5 Adicione este dicionário a uma lista;
- 6 Pergunte se o usuário deseja cadastrar outro aluno;
- 7 Continue repetindo o processo enquanto a resposta for "S";
- 8 Ao final, exiba todos os alunos cadastrados.

TENTE! ESTOU DE OLHO, HEIN?!

Solução

```
alunos = []
```

```
while True:
```

```
    nome = input("Nome do aluno: ")
```

```
    idade = int(input("Idade: "))
```

```
    cidade = input("Cidade: ")
```

```
    aluno = {
```

```
        "nome": nome,
```

```
        "idade": idade,
```

```
        "cidade": cidade
```

```
    }
```

```
    alunos.append(aluno)
```

```
    continuar = input("Deseja cadastrar outro aluno? (S/N): ").upper()
```

```
    if continuar == "N":
```

```
        break
```

```
print()

print("==== ALUNOS CADASTRADOS ====")

for aluno in alunos:

    print()

    print("Nome:", aluno["nome"])

    print("Idade:", aluno["idade"])

    print("Cidade:", aluno["cidade"])
```

Durante o desafio, utilizamos praticamente todos os conceitos apresentados nesta apostila.

- O laço while para manter o programa em execução durante os cadastros;
- O if para verificar quando o usuário deseja encerrar o programa;
- O dicionário para armazenar as informações de cada aluno;
- A lista para reunir todos os cadastros realizados;
- O laço for para percorrer a lista e exibir o relatório final;
- Conversão de tipos utilizando int() para a idade.

Perceba como um conceito complementa o outro. Sozinhos, eles resolvem pequenos problemas.

Juntos, permitem criar aplicações muito mais completas. Igual a gente, estudando juntos!

Se você quiser melhorar ainda mais esse desafio, implemente melhorias para informar quantos alunos foram cadastrados utilizando len(); Exibir apenas os alunos maiores

de 18 anos; Permitir pesquisar um aluno pelo nome; Criar uma lista contendo apenas as cidades cadastradas; Utilizar um conjunto (set) para exibir apenas as cidades sem repetição.

AGRADECIMENTO

Se você chegou até aqui, quero te dizer: obrigada.

De verdade.

Finalizar um material assim não é só sobre aprender conteúdo. É sobre dedicação, tempo e vontade de evoluir. E só de você estar aqui, já mostra muito sobre você.

Essa apostila foi criada com um propósito muito claro: mostrar que conhecimento não precisa ser complicado. Que assuntos que parecem difíceis podem, sim, ser entendidos quando explicados da forma certa.

E mais do que isso: que conhecimento deve ser para todos.

Por isso, este material também foi pensado para ser acessível. Ele conta com versão em Braille, porque inclusão não é um diferencial — é o mínimo.

Todo mundo merece aprender. Todo mundo merece ter acesso.

Se, de alguma forma, esse conteúdo te ajudou a entender melhor, a se organizar ou até a enxergar as coisas de um jeito diferente, então ele já cumpriu o seu papel.

Obrigada por fazer parte disso.

E lembre-se: você não precisa complicar para fazer dar certo.

CONTATO

Se você quiser continuar aprendendo, trocar ideias ou acompanhar mais conteúdos sobre projetos, tecnologia e organização no dia a dia, você pode me encontrar por aqui:

Instagram: @CAROL_G_INTECH

LinkedIn: Caroline Gonçalves

Site: <https://cpgconsulting.com.br/>

E-mail: cpgtechconsulting@gmail.com

Fique à vontade para se conectar ou mandar uma mensagem. Vou adorar continuar essa troca com você.